

Data Structures

Assigned Date: 30-08-2026

Due Date: 10-09-2026

Instructor(s)

Mr. Muhammad Naveed

Assignment # 01

Course code: CS2001

Total marks: 160

Questions: 5

Important Instructions:

1. Put your .cpp and .doc/.pdf files into a zip folder and upload it onto the google classroom submission folder. Name your solution file with your roll number, i.e., Assignment1_24L_1111.zip. Assignment in any other format (extension) will not be accepted and will be awarded with zero marks.
2. You are not allowed to copy solutions from other students. We will check your solution for plagiarism using plagiarism checkers. If any sort of cheating is found, negative marks will be given to all students involved.
3. Late submission of your solution is not allowed

CLO 01: Demonstrate basic concepts of data structures and algorithms

Q1: Give an estimate of $T(N)$ for each line of the code. Run the code for different values of N to prove your time complexity. Also compute the big-O **[Marks 5*8=40]**

Note: You are required to show complete steps

Questions	Time Complexity (Big O)
1. <pre>int count=0; for (int i=1;i<n;i*=3){ for (int j=0;j<i;++j){ count++; } cout << count << endl;</pre>	

National University of Computer and Emerging Sciences
Lahore Campus

<pre>} </pre>	
2. <pre>int i=1; while(i<=n) { int j=1; while(j<=i) { cout<<"."; j=j*2; } i=i*2; } </pre>	
3. <pre>void mystery(int n) { for (int i = 1; i <= n; i *= 2) { for (int j = i; j <= n; j += i) { int k = j; while (k > 0) { k /= 2; } } } } </pre>	
4. <pre>int i=1; while(i<=n) { int j=1; while(j<=i) { cout<<"."; } } </pre>	

National University of Computer and Emerging Sciences
Lahore Campus

<pre> j=j*2; } i=i*2; } </pre>	
5. <pre> void bubbleSort(int arr[], int n) { for (int i = 0; i < n - 1; i++) { for (int j = 0; j < n - i - 1; j++) { if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); } } } } </pre>	
6. <pre> sum=0; for(i=0;j<=n;i++) { for(j=0;j<i*i;j++){ for(k=0;k<j;k++){ sum++; } } } </pre>	
7. <pre> void mystery(int n) { for (int i = 1; i <= n; i *= 2) { int x = 1; while (x <= n) { for (int j = 1; j <= x; j *= 2) { cout << "*"; } } } } </pre>	

National University of Computer and Emerging Sciences
Lahore Campus

<pre> } x *= 2; } } }</pre>	
8. <pre>int sqrt(int N) { int d; for(d=0; d*d<=N; d++) {} return d-1; } bool primeNumber(int n) { bool isPrime = true; for (int d=2; d<= sqrt(n) ;++d) { if (n%d==0) return false; } return true; }</pre>	

National University of Computer and Emerging Sciences
Lahore Campus

Q2: Give the worst, average and best case time complexity in Big O notation. State space complexity as well. **[Marks 20]**

1. <pre>void mystery(int n) { int** a = new int*[n]; for (int i = 0; i < n; i++) { a[i] = new int[i * i + 1]; for (int j = 0; j < i; j++) { int* temp = new int[j + 1]; for (int k = 0; k <= j; k++) temp[k] = k; delete[] temp; } } }</pre>	
2. <pre>void mystery(int n) { int size = 1; for (int i = 0; i < n; i++) { size *= 2; } int* arr = new int[size]; }</pre>	

3.

```
void fun(int n) {  
    int* arr = new int[n];  
    for (int i = 0; i < n; i++)  
        arr[i] = i;  
  
    for (int i = 0; i < n; i++) {  
        int* row = new int[i + 1];  
        for (int j = 0; j <= i; j++)  
            row[j] = arr[i] + j;  
        // row is NOT deleted here  
        arr[i] = row[0];  
    }  
  
    int* temp = new int[n * n];  
    delete[] temp;  
}
```

National University of Computer and Emerging Sciences

Lahore Campus

Important Instructions (apply to all questions below):

1. You must design your **own custom class/struct** for each node. It will contain fields specific to the scenario — not a generic **data + next** node.
2. You must add **your own member functions** to solve each part. Function names given below are the *minimum required*; you may add helper functions.
3. You are **not allowed** to use built-in list/array/queue containers (e.g., `std::list`, `vector` of nodes) to store the chain itself. The linking must be done through raw node pointers (`next`, and `prev/circular` links where applicable).
4. Each question is broken into parts. Read carefully. Submit working, tested code for every part — later parts build on earlier parts.
5. Also make appropriate print functions for every list class that shows all the nodes in that list.

Q3: Singly Linked List [Marks 30]:

Scenario: University Course Waitlist Manager

The Registrar's Office wants software to manage the waitlist for a single course section using a **singly linked list**. Every waitlisted student is a node containing:

- `studentID` (unique)
- `cgpa` (float — determines priority)
- `requestTime` (integer — smaller value means the student joined the waitlist earlier)
- `creditHoursEnrolled` (credit hours the student is *already* registered for in other courses)
- `skipCount` (integer, starts at 0 — tracks how many times this student was passed over)
- `next`

Priority rule: higher `cgpa` = higher priority. If two students have equal `cgpa`, the one with the smaller `requestTime` (earlier request) has higher priority.

Eligibility rule: this course is worth 3 credit hours. A student can only be allocated a seat if `creditHoursEnrolled + 3 <= 18`. Otherwise, they are *not* eligible yet.

Part A — Ordered Insertion

Implement `insertStudent(studentID, cgpa, requestTime, creditHoursEnrolled)`

National University of Computer and Emerging Sciences

Lahore Campus

which inserts the new student into the correct position in the linked list according to the **priority rule** above (the list must always remain sorted by priority).

Part B — Seat Allocation with Skip/Auto-Drop

Implement `allocateSeat()`. When a seat opens up:

- Traverse from the head.
- For every **ineligible** student encountered, increment their `skipCount` by 1 and move on.
- If a student's `skipCount` reaches **3**, they are automatically removed from the waitlist entirely (print an auto-drop notification) — this happens *the moment* the count hits 3, even mid-traversal.
- The **first eligible** student found is removed from the list and the seat is allocated to them (print an allocation message).
- If no eligible student exists in the whole list, print that the seat remains unallocated.

Part C — Merging Two Waitlists

A parallel section of the same course gets cancelled.

Implement `mergeWaitlists(Node* otherHead)` which merges that section's waitlist into this one, producing a **single list still sorted by the priority rule** (same tie-break). You must do this by **relinking existing nodes** (no new node creation, no copying into an array and rebuilding).

Examples:

Insertions (in this order) into List A:

```
insertStudent(S1, cgpa=3.5, requestTime=1, creditHoursEnrolled=15)
```

```
insertStudent(S2, cgpa=3.8, requestTime=2, creditHoursEnrolled=16)
```

```
insertStudent(S3, cgpa=3.8, requestTime=1, creditHoursEnrolled=12)
```

```
insertStudent(S4, cgpa=3.2, requestTime=3, creditHoursEnrolled=9)
```

Resulting order (head → tail): `S3(3.8) → S2(3.8) → S1(3.5) → S4(3.2)` (*S3 beats S2 despite equal CGPA because S3's requestTime is earlier.*)

Now call `allocateSeat()`:

- S3: `12+3=15 <= 18` → eligible → **allocated to S3**.

National University of Computer and Emerging Sciences

Lahore Campus

List becomes: $S2 \rightarrow S1 \rightarrow S4$

Separate example for skip/auto-drop behavior, given this list:

$S7$ (creditHoursEnrolled=17, skipCount=0) $\rightarrow$ $S8$ (creditHoursEnrolled=16, skipCount=2) $\rightarrow$ $S9$ (creditHoursEnrolled=6, skipCount=0)

Call `allocateSeat()`:

$S7$: $17+3=20 > 18 \rightarrow$ ineligible $\rightarrow$ skipCount becomes 1

$S8$: $16+3=19 > 18 \rightarrow$ ineligible $\rightarrow$ skipCount becomes 3 $\rightarrow$ AUTO-DROPPED, print notification

$S9$: $6+3=9 \leq 18 \rightarrow$ eligible $\rightarrow$ ALLOCATED to $S9$

Final list: $S7$ (skipCount=1) — head, alone ($S8$ dropped, $S9$ allocated/removed).

Q4: Circular Linked List [Marks 40]:

Scenario: AI Customer Support Bot Rotation

A company routes live-chat customers to AI support bots arranged in a **circular singly linked list**. Each node:

- `botID`
- `currentLoad` (active chats it's handling)
- `maxLoad` (capacity)
- `isResting` (bool, starts false)
- `chatsHandledSinceRest` (int, starts 0)
- `lapsPassedWhileResting` (int, starts at 0)
- `next`

There is a `current` pointer indicating whose "turn" it is next. It should be static to all list objects made.

Assume a threshold $R = 5$.

National University of Computer and Emerging Sciences

Lahore Campus

Part A — Bot Onboarding (Building the Ring)

Before any query can be routed, the ring itself has to be built — and it keeps growing as the company adds more bots over time, so insertion has to work correctly at every ring size, not just as a one-time setup step.

Implement `addBot(botID, maxLoad)`:

- A newly onboarded bot always starts with `currentLoad = 0`, `isResting = false`, `chatsHandledSinceRest = 0`.
- If the ring is currently **empty**, the new bot becomes the only node: its `next` must point to **itself** (a valid 1-node circle), and `current` is initialized to point to it.
- If the ring already has one or more bots, the new bot is inserted **immediately after whichever bot `current` is pointing at** — i.e., it becomes the very next bot in line after whoever is currently serving. This means insertion position depends on where `current` happens to be at the time, not simply at the end. `current` itself does **not** move as a result of an insertion.
- In every case, the circular structure must remain valid afterward: traversing `next` starting from any node must eventually return to that same node, no matter how many bots have been added.

Part B — Round-Robin Assignment

Implement `assignQuery()`:

- Start at `current`.
- If that bot `isResting`, skip it (don't assign, don't count it) and check the next bot.
- If it is *not* resting and `currentLoad < maxLoad`, assign the query here: `currentLoad++`, `chatsHandledSinceRest++`. Move `current` to the next bot for the next call.
- If it is not resting but already full (`currentLoad == maxLoad`), skip it and keep moving circularly.
- If a **full circle** is completed with no bot available, print "System Overload — query queued" and take no action.

Part C — Rest Cycle

Implement `rotateCycle()`. It traverses over all bots and whenever a bot's `chatsHandledSinceRest` reaches a threshold `R`, immediately sets `isResting = true` on that bot (it keeps its current chats but stops accepting new ones). At no time should there be a bot whose `chatsHandledSinceRest > R`.

National University of Computer and Emerging Sciences

Lahore Campus

Every time the `current` pointer passes a resting bot, that bot's `lapsPassedWhileResting` is incremented by 1. One **full lap** back around to a resting bot for the second time (i.e., it has `lapsPassedWhileResting = 2`), that bot's rest ends: reset `isResting = false` and `chatsHandledSinceRest = 0`, rejoining active rotation.

Part D — Crash & Cascade Forwarding

Implement `crashBot(botID)`. When a bot crashes:

- All of its `currentLoad` chats must be forwarded, starting with the *next* bot in the circle.
- Each subsequent bot absorbs as many chats as it has room for (up to its `maxLoad`, ignoring its resting status — crash forwarding is an emergency override).
- Any chats that don't fit continue circularly to the bot after that, and so on.
- If a full circle passes and chats still remain unplaced, print them as "unassigned."
- The crashed bot's node is permanently removed from the circular list, with the circular structure kept intact. If `current` is pointed at the crashed bot, move it to the next surviving node.

Example — Building the Ring (Part A)

Starting from a completely empty system:

`addBot(B1, 5)` - ring empty → B1 self-loops, `current = B1`

`addBot(B2, 4)` - inserted right after `current` (B1) → B1 → B2 → B1, `current` still B1

`addBot(B3, 3)` - inserted right after `current` (B1) → B1 → B3 → B2 → B1, `current` still B1

Note the order: `B3` ends up *before* `B2` in the ring, even though `B2` was added first — because both were inserted right after `current`, and `current` never moved. This is the behavior your `addBot` must reproduce.

Example — Assignment & Crash (Parts B–D)

Suppose a Ring (in order):

`B1(load=2,max=5,resting=F,handled=3) → B2(load=5,max=5,resting=F,handled=1) →`

`B3(load=1,max=4,resting=T,handled=0) → B4(load=0,max=3,resting=F,handled=0) → (back to B1) current = B1`

Two incoming queries:

National University of Computer and Emerging Sciences

Lahore Campus

assignQuery(): current=B1, not resting, $2 < 5 \rightarrow$ ASSIGN. load $2 \rightarrow 3$, handled $\rightarrow 4$. current moves to B2.

assignQuery(): current=B2, full ($5 == 5$) $\rightarrow$ skip $\rightarrow$ B3 resting $\rightarrow$ skip $\rightarrow$ B4, $0 < 3 \rightarrow$ ASSIGN. load $0 \rightarrow 1$. current moves to B1.

Crash example (separate scenario): B2 crashes holding 5 active chats. Next in ring is B3 (load=1, max=4) $\rightarrow$ absorbs 3 chats to reach its cap (load $1 \rightarrow 4$, full). Remaining 2 chats forward to B4 (load=1, max=3) $\rightarrow$ absorbs both (load $1 \rightarrow 3$, full). All 5 chats placed. B2's node is removed; ring is now $B1 \rightarrow B3 \rightarrow B4 \rightarrow B1$.

Q5: Doubly Linked List [Marks 30]:

Scenario: Smart Thermostat Schedule Timeline

A smart home app stores a homeowner's daily temperature schedule as a **doubly linked list**, always kept sorted by start time. Each node:

- **startTime** (minutes after midnight, 0–1439)
- **duration** (minutes)
- **targetTemp**
- **isLocked** (bool — a "do not auto-modify" flag set by the user)
- **prev, next**

Assume an overlap-merge threshold $T = 10$ minutes.

Part A — Sorted Insertion

Implement **insertSchedule(startTime, duration, targetTemp, isLocked=false)**, inserting the node at the correct chronological position and correctly updating **both prev** and **next** links of its new neighbors.

Part B — Overlap Detection and Cascading Merge

After every insertion, check the new node against its immediate neighbor(s):

- An event occupies **[startTime, startTime + duration)** time.
- If the new event's end time extends past the next node's start time by **more than T minutes**, they must **merge** into one node: **startTime = min(both starts)**, **duration**

National University of Computer and Emerging Sciences

Lahore Campus

spans to $\max(\text{both end times})$, $\text{targetTemp} = \text{rounddown}(\text{average of both temps})$, $\text{isLocked} = \text{true}$ if either was locked. Remove both original nodes and splice in the single merged node, correctly reconnecting prev/next on both sides.

- If the overlap is T minutes or less, don't merge — just print a "minor overlap" warning and leave both nodes as-is.
- Do this check on **both sides** (previous neighbor and next neighbor), and if a merge happens, **re-check the merged node against its new neighbors again** — a single insertion can trigger a chain of merges.

Part C — Cursor-Based Smart Undo

The system keeps a **cursor** pointer (initially at head) that the user can move with $\text{next()}/\text{previous()}$ calls while browsing their schedule. Implement $\text{undoLast}(n)$:

- Starting from **cursor**, move **backward** using prev pointers.
- Collect up to n **unlocked** nodes to delete. Any **locked** node encountered along the way is skipped (left in place) and does *not* count toward n .
- If the traversal reaches the head ($\text{prev} == \text{NULL}$) before finding n unlocked nodes, delete however many were found and print how many of the n requested undos were actually completed, and how many locked nodes were skipped.
- Deletions must correctly relink the remaining neighbors' prev/next pointers.

Example

Insert in this order:

- $\text{insertSchedule}(420, 30, 22, \text{false})$ // 7:00 AM, 30 min, 22°C
- $\text{insertSchedule}(1080, 60, 20, \text{false})$ // 6:00 PM, 60 min, 20°C
- $\text{insertSchedule}(450, 45, 23, \text{false})$ // 7:30 AM, 45 min, 23°C → no overlap, list now: $[420, 450, 1080]$
- $\text{insertSchedule}(435, 50, 25, \text{false})$ // 7:15 AM, 50 min, 25°C

The 4th insert (435, ends 485) lands between the 420 node (ends 450) and the 450 node.

- Overlap with previous (420–450): $450 - 435 = 15 > 10 \rightarrow$ **merge**. Merged node: $\text{start}=420, \text{end}=485, \text{duration}=65, \text{temp}=\text{round}((22+25)/2)=23, \text{locked}=\text{false}$.
- Re-check this merged node (420–485) against its new next neighbor, the 450 node (450–495): $\text{overlap} = 485 - 450 = 35 > 10 \rightarrow$ **cascading merge again**. New merged node: $\text{start}=420, \text{end}=495, \text{duration}=75, \text{temp}=\text{round}((24+23)/2)=23, \text{locked}=\text{false}$.

National University of Computer and Emerging Sciences

Lahore Campus

- Re-check against next neighbor 1080: no overlap. Done.

Final list: [420, dur=75, 24°C] → [1080, dur=60, 20°C]

Now insert one more, locked, event: `insertSchedule(900, 30, 19, true)` → no overlap either side. List: [420,75,24°] → [900,30,19°,locked] → [1080,60,20°]

Suppose `cursor` is currently at the 1080 node. Call `undoLast(2)`:

prev of 1080 → 900 node → locked → skip (doesn't count)

prev of 900 → 420 node → unlocked → mark for deletion (count = 1)

prev of 420 → NULL (head reached) → stop

Output: "Undid 1 of 2 requested changes (reached start of timeline; 1 locked entry was skipped)."

Final list: [900,30,19°,locked] → [1080,60,20°]
